

ダッシュボードのスナップショット — 機能設計

003_DD-FN — 003_DD と 003_DD-API が使う。要件 REQ-028～REQ-042。

改版履歴

項目	値
ドキュメントID	003_DD-FN
システム名	ProductionManagementAI
サブシステム名	製造指示（Production orders）
作業項目	WI-004
作成者	Claude（ThanhTN 向け）
作成日	2026-09-22
最終更新者	Claude（ThanhTN 向け）
最終更新日	2026-09-22

版	日付	作成者	改版内容
1	2026-09-22	Claude（ThanhTN 向け）	新規作成
2	2026-09-22	Claude（ThanhTN 向け）	\$6 <code>SystemHealthService.CheckAsync</code> と <code>IDatabasePing</code> （DEC-017、DEC-020）。\$7 ヘルスのパスで延長しないルール（DEC-019）。可観測性を拡張
3	2026-09-22	Claude（ThanhTN 向け）	実装との整合（WI-004 DEC-024）：\$3 は <code>SqlQuery<T></code> の代わりに、EF の接続とトランザクション上の ADO.NET コマンドで文を実行する。ヘルスのコードは <code>Health</code> 名前空間
4	2026-09-22	Claude（ThanhTN 向け）	\$7：セキュリティスタンプの再検証の後も延長を抑止（WI-004 DEC-025、TC-225 で発見）

概要とメソッドの索引

項目	値
モジュール名	DashboardService とそのリーダーのポート（ ProductionManagementAI.Application.Dashboard 、リーダーの実装は ProductionManagementAI.Infrastructure.Dashboard ）
概要	「今」をダッシュボードの数値に変える：工場ローカルの日付からすべての期間を導き、003_DB の 7 つの文を 1 つのスナップショットで読み、結果を整える。ProductionOrderService とは別のサービスで、共有するユースケースはない — 共有するのは時計、テレメトリーの源、結果の慣例だけで、それらは再利用する

No	メソッド名	概要	備考
1	DashboardService.GetSnapshotAsync	時計を一度読み、期間を組み立て、読み、整え、テレメトリーを記録	API-DSH-01
2	DashboardWindow.For	数値が依存するすべての日付と時刻を T から — 純粋関数	新規
3	IDashboardReader.ReadAsync	1 つの REPEATABLE READ READ ONLY トランザクションでの 003_DB Q1～Q6	新しいポート。Infrastructure の DashboardReader
4	DashboardMapper.ToResponse	生の行 → DashboardResponse : 0 での埋め、区分の順序、リードタイムの丸め	新規、純粋
5	IPlantClock.DateOf / StartOfDayUtc / TimeZoneId	期間が必要とする工場ローカルの暦の変換	既存のポート（001_DD-FN \$6）への追加のメンバー
6	SystemHealthService.CheckAsync	2 秒のタイムアウトでデータベースに ping	API-SYS-01。ポート IDatabasePing 、Infrastructure の DatabasePing
7	Cookie の OnCheckSlidingExpiration	ヘルスのリクエストはセッションを延長しない	DependencyInjection.cs

共通のユーティリティの参照

No	名前	概要	備考
1	IPlantClock	工場ローカルの日付とタイムゾーン	既存のポート。3 つのメンバーを追加（\$5）。Today と CurrentYear は変更なし
2	TimeProvider	スナップショットごとに 1 回だけ読む「今」	既存。テストでは FakeTimeProvider で固定
3	ProductionOrderTelemetry	ProductionManagementAI.ProductionOrders という名前の ActivitySource + Meter	既存。カウンターを 1 つ追加（可観測性を参照）。再利用するので Program.cs は新しい源やメーターを登録しない
4	ILogger<DashboardService>	構造化ログ LoggerMessage のソース生成	

No	名前	概要	備考
5	AppDbContext	EF Core のコンテキスト。リーダーは Database.SqlQuery<T> (FormattableString) を使う	補間した値はバインドされたパラメータになり、SQL に連結しない (ai/rules/database.md)

リクエストのデータ

データベースに送るパラメータ — ちょうど 003_DB のパラメータの表で、DashboardWindow (§2) が作る。すべてバインドされたパラメータ。

No	名前	変数名	型	長さ	使用するメソッド	備考
1	工場の今日	Today	DateOnly	—	Q2、Q3	T
2	納期間近の終わり	SoonEnd	DateOnly	—	Q3b	T + 7 (DEC-007)
3	第 0 週	Week0	DateOnly	—	Q2	W(T)、T 以前で最も近い月曜日
4	推移の開始	TrendStartUtc	DateTimeOffset	—	Q5、Q6	工場の日 W(T) – 77 の開始
5	週の開始	WeekStartUtc	DateTimeOffset	—	Q5	工場の日 W(T) の開始
6	月の開始	MonthStartUtc	DateTimeOffset	—	Q5	T の月の工場の 1 日の開始
7	期間の開始	WindowStartUtc	DateTimeOffset	—	Q5	工場の日 T – 29 の開始
8	終わり	EndUtc	DateTimeOffset	—	Q5、Q6	工場の日 T + 1 の開始
9	タイムゾーン	TimeZoneId	string	—	Q5、Q6	PlantOptions.TimeZone、起動時に検証

レスポンス／値の加工

DashboardResponse (項目の一覧は 003_DD-API) は、リーダーの結果 DashboardRaw から DashboardMapper.ToResponse (§4) が組み立てる：

No	名前	変数名	型	長さ	必須	値の加工	備考
1	ステータス別件数	StatusCounts	(string status, int count)[]	4 以内	yes	ないステータス → 0。Total = 合計	Q1
2	作業量の行	Workload	(int bucket, int count, long quantity)[]	10 以内	yes	bucket – 1 → overdue、0～7 → 第 k week、8 → later。ない区分 → 0	Q2

No	名前	変数名	型	長さ	必須	値の加工	備考
3	納期遅れ／納期間近の行	Overdue 、 DueSoon	(DashboardOrderRow[] rows, int total)	10 行以内	yes	total は最初の行の count(*) OVER () から。行がなければ 0	Q3a、Q3b
4	上位の製品	TopProducts	TopProductRow[]	10 以内	yes	返ったまま（すでに並べ替え済み）	Q4
5	納品実績の行	Delivery	DeliveryRow	1	yes	NULL の合計 → 0。NULL の平均 → null	Q5
6	推移の行	Trend	(DateOnly weekStart, int count)[]	12 以内	yes	想定する 12 月の月曜日に並べる。ない → 0	Q6

メソッドの設計

1. DashboardService.GetSnapshotAsync

項目	値
説明	現在の時点の完全なダッシュボードのスナップショットを組み立てる
戻り値の型	Task<DashboardResponse>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	CancellationToken	cancellationToken	

戻り値

型	名前	説明
DashboardResponse	snapshot	常に完全。このメソッドに業務上の失敗はない — 拒否する入力も、見つけるものもない — ので、Result<T> ではなくレスポンスを直接返す。想定外の例外は 002_DD-FN と同じく全体の Problem Details のハンドラーに伝える（500、MSG-E013）

処理概要：時計は一度だけ読み、T、すべての期間、asOf の時刻、レスポンスの today はすべてその 1 回の読み取りから導く。そのため、日付をまたいで計算したダッシュボードが 2 つの「今日」を混ぜることはない。

処理フロー

ステップ	説明	呼び出し
1	スパン <code>ProductionOrder.Dashboard</code> を開始	<code>ProductionOrderTelemetry.Source</code>
2	<code>utcNow = timeProvider.GetUtcNow()</code> 、 <code>today = plantClock.DateOf(utcNow)</code>	<code>TimeProvider</code> 、 <code>IPlantClock</code> (\$5)
3	<code>window = DashboardWindow.For(today, plantClock)</code>	\$2
4	<code>raw = await reader.ReadAsync(window, ct)</code>	\$3
5	<code>response = DashboardMapper.ToResponse(window, raw, utcNow, plantClock.TimeZoneId)</code>	\$4
6	スパンにタグ付け（ <code>dashboard.today</code> 、 <code>dashboard.active_orders = draft + inProgress</code> 、 <code>dashboard.window_completed = onTime.completedCount</code> ）。カウンター <code>dashboard_loaded{outcome=success}</code> 。返す	テレメトリー
7	例外時：カウンター <code>dashboard_loaded{outcome=error}</code> 、スパンのステータス <code>Error</code> 、例外付きのログ <code>DashboardSnapshotFailed</code> 、再スロー	テレメトリー、ロガー

2. `DashboardWindow.For`

項目	値
説明	003_BD D-01～D-09 と 003_DB Q1～Q6 が依存するすべての日付と UTC の時刻を、工場ローカルの日付 T から計算する
戻り値の型	<code>DashboardWindow</code> （不変のレコード）
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>DateOnly</code>	<code>today</code>	T
2	<code>IPlantClock</code>	<code>clock</code>	<code>StartOfDayUtc</code> と <code>TimeZoneId</code> のためだけ

戻り値

型	名前	説明
DashboardWindow	window	リクエストのデータの 9 項目に加え、MonthStart（日付）、WindowStart（T - 29）、TrendWeeks（12 個の月曜日 W(T) - 77 ... W(T)）

処理概要：T とタイムゾーンの純粋関数。暦のルールはすべてここで一度だけ定めるので、データベースなしで、すべての曜日と月の境界について単体テストできる（TC-205、TC-209）。

処理フロー

ステップ	説明	呼び出し
1	Week0 = today.AddDays(-(7 + (int)today.DayOfWeek - 1) % 7)) — T 以前で最も近い月曜日（日曜日は 6 日前に対応）	—
2	SoonEnd = today + 7、WindowStart = today - 29、MonthStart = new DateOnly(today.Year, today.Month, 1)	—
3	TrendWeeks = [Week0 - 77, Week0 - 70, ..., Week0]（12 個の月曜日）	—
4	日付の各下限を、その工場の日の開始の UTC に変換：TrendStartUtc = StartOfDayUtc(TrendWeeks[0])、WeekStartUtc = StartOfDayUtc(Week0)、MonthStartUtc、WindowStartUtc、EndUtc = StartOfDayUtc(today + 1)	IPlantClock.StartOfDayUtc
5	TimeZoneId = clock.TimeZoneId	—

3. IDashboardReader.ReadAsync（DashboardReader）

項目	値
説明	003_DB Q1～Q6（7 つの文）を 1 つの読み取り専用のスナップショットのトランザクションの中で実行する
戻り値の型	Task<DashboardRaw>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	DashboardWindow	window	パラメータ
2	CancellationTok	cancellationTok	

戻り値

型	名前	説明
DashboardRaw	raw	整える前の 7 つの結果

処理概要：生 SQL を使う。Q2 の区分の式、Q3 のウィンドウの件数、Q5 の `FILTER` の集計、Q6 の `date_trunc ... AT TIME ZONE` には素直な LINQ の翻訳がないためである。各文は 003_DB のテキストで、すべての値を `NpgsqlParameter` にして EF の接続とトランザクション上の ADO.NET コマンドとして実行し（WI-004 DEC-024）、小さな行のレコード（`StatusCountRow`、`WorkloadRow`、`DashboardOrderRow`、`TopProductRow`、`DeliveryRow`、`TrendRow`）に序数で読み込む。文は 003_DB がレビューした SQL そのもので、LINQ による近似ではない。

処理フロー

ステップ	説明	呼び出し
1	<code>await using var tx = await db.Database.BeginTransactionAsync(IsolationLevel.RepeatableRead, ct)</code>	EF Core / Npgsql
2	<code>SET TRANSACTION READ ONLY</code> — PostgreSQL が要求するとおり、どのクエリよりも前のトランザクションの最初の文。このトランザクションでの書き込みはすべて失敗するようになり、データベースでダッシュボードの「データを変更しない」（REQ-039）を守る	<code>ExecuteSqlRawAsync</code> （定数のテキスト、入力なし）
3	Q1 ステータス別件数	<code>SqlQuery<StatusCountRow></code>
4	<code>Today</code> 、 <code>Week0</code> での Q2 作業量の区分	<code>SqlQuery<WorkloadRow></code>
5	Q3a 納期遅れ（ <code>due_date < Today</code> ）、Q3b 納期間近（ <code>due_date BETWEEN Today AND SoonEnd</code> ）	<code>SqlQuery<DashboardOrderRow></code> × 2
6	Q4 上位の製品	<code>SqlQuery<TopProductRow></code>
7	5 つの時刻とタイムゾーンでの Q5 納品実績の行	<code>SqlQuery<DeliveryRow></code>
8	<code>TrendStartUtc</code> 、 <code>EndUtc</code> 、タイムゾーンでの Q6 推移	<code>SqlQuery<TrendRow></code>
9	<code>await tx.CommitAsync(ct)</code> （読み取り専用のコミットでスナップショットを解放）し、 <code>DashboardRaw</code> を返す	—

文は 1 つの接続で順番に実行する。トランザクションはそれらを並列に実行できず、003_DB の件数では合計で数ミリ秒である。Npgsql のコマンドタイムアウトは 002_DD と同じく既定の 30 秒のまま。

4. DashboardMapper.ToResponse

項目	値
説明	生の行を API のレスポンスに整える
戻り値の型	DashboardResponse

項目	値
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	DashboardWindow	window	区分の日付と期間の開始
2	DashboardRaw	raw	リーダーの出力
3	DateTimeOffset	asOf	1 回だけの時計の読み取り
4	string	timeZoneId	timeZone としてそのまま返す

戻り値

型	名前	説明
DashboardResponse	response	003_DD-API を参照

処理概要：純粋関数。「ないものは 0」のルールと、サーバーが持つ唯一の丸めのルールはすべてここにあるので、データベースなしで単体テストできる。

処理フロー

ステップ	説明	呼び出し
1	ステータス別件数：4 つの各ステータスを Q1 の行で探し、なければ 0。total = その合計	—
2	作業量：表示順にちょうど 10 件を組み立てる — overdue（区分 -1）、k = 0...7 の 8 個の week（weekStart = Week0 + 7k、ただし k = 0 では Today。weekEnd = Week0 + 7k + 6）、その後 later（区分 8）。それぞれその区分の Q2 の件数と数量を取り、なければ 0	—
3	納期遅れ／納期間近：total = 最初の行のウィンドウの件数、行がなければ 0。orders = DashboardOrder に対応付けた行	—
4	上位の製品：返ったまま対応付ける	—
5	今週／今月の完了：件数、NULL の合計 → 0、from = Week0 / MonthStart	—
6	期限内：{ onTimeCount = x, completedCount = y, windowStart = WindowStart }	—
7	リードタイム：averageDays = y == 0 ? null : Math.Round(avg, 1, MidpointRounding.AwayFromZero)、orderCount = y	—

ステップ	説明	呼び出し
8	推移：12 個の各 <code>TrendWeeks</code> について、 <code>week_start</code> が一致する Q6 の件数、なければ 0。 <code>weekEnd = weekStart + 6</code>	—
9	<code>asOf</code> 、 <code>today</code> 、 <code>timeZone</code>	—

整合性の確認は実行時ではなく単体テストで行う：作業量の件数の合計は `draft + inProgress` に等しく、Q3a の合計は `overdue` の区分の件数に等しい。どちらも 1 つのスナップショットから導かれる（DEC-015）。

5. `IPlantClock` — 追加のメンバー

項目	値
説明	ダッシュボードの暦が必要とする 3 つのメンバー。 <code>Today</code> と <code>CurrentYear</code> を変えずに既存のポートに追加する
戻り値の型	下を参照
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数／戻り値

メンバー	シグネチャ	動作
<code>DateOf</code>	<code>DateOnly DateOf(DateTimeOffset utc)</code>	ある時刻の工場ローカルの日付。 <code>Today</code> は <code>DateOf(timeProvider.GetUtcNow())</code> になるので、2 つが食い違うことはない
<code>StartOfDayUtc</code>	<code>DateTimeOffset StartOfDayUtc(DateOnly date)</code>	その工場ローカルの日付が始まる UTC の時刻：ローカルの 0 時を <code>TimeZoneInfo</code> で変換したもの。夏時間のずれでローカルの 0 時が存在しない場合は、その日の最初の有効なローカル時刻を使う。 <code>Asia/Tokyo</code> には夏時間がない。このルールは工場のタイムゾーンを設定し直しても正しく保つためにある
<code>TimeZoneId</code>	<code>string TimeZoneId { get; }</code>	設定した IANA の ID（ <code>PlantOptions.TimeZone</code> ）、起動時に検証。PostgreSQL に <code>@zone</code> として渡す：PostgreSQL と .NET はどちらも IANA の <code>tzdata</code> を読むので、タイムゾーンのルールで一致する

処理概要：`PlantClock` はすでに `TimeZoneInfo` を一度解決している。これらのメンバーはそれを再利用するので、他のクラスがタイムゾーンを変換することはない。

6. `SystemHealthService.CheckAsync`

項目	値
説明	データベースが 2 秒以内に <code>SELECT 1</code> に応答するかを報告する

項目	値
戻り値の型	<code>Task<SystemHealthResponse></code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>CancellationToken</code>	<code>cancellationToken</code>	リクエストのトークン

戻り値

型	名前	説明
<code>SystemHealthResponse</code>	<code>health</code>	<code>{ Database = "ok" \ "unavailable", CheckedAt }</code> — ping の失敗で例外にすることはない

処理概要：ポート `IDatabasePing.PingAsync(ct)` は `DatabasePing` が `db.Database.ExecuteSqlRawAsync("SELECT 1", ct)`（定数のテキスト、入力なし）として実装する。サービスはリクエストのトークンを 2 秒の `CancellationTokenSource` と結び付ける。ポートがあるのは、共有のデータベースのコンテナを止めずに、結合テストがそれを失敗する実装や遅い実装に置き換えられるようにするためである（TC-224）。

処理フロー

ステップ	説明	呼び出し
1	スパン <code>System.Health</code> を開始	<code>ProductionOrderTelemetry.Source</code>
2	<code>using var timeout = CancellationTokenSource.CreateLinkedTokenSource(ct); timeout.CancelAfter(2 s)</code>	—
3	<code>await ping.PingAsync(timeout.Token) → database = "ok"</code>	<code>IDatabasePing</code>
4	（ <code>ct</code> ではなく） タイムアウトによる <code>OperationCanceledException</code> 、または接続からの <code>DbException</code> / <code>NpgsqlException</code> / <code>InvalidOperationException</code> → <code>database = "unavailable"</code> 。例外の型だけを含む <code>Warning</code> のログ <code>DatabasePingFailed</code> 。スパンのステータス <code>Error</code>	ロガー
5	<code>ct</code> 自体が取り消された（クライアントがいなくなった）場合は再スロー — 報告するものはない	—
6	カウンター <code>health_checks{database}</code> 。 <code>{ database, CheckedAt = timeProvider.GetUtcNow() }</code> を返す	テレメトリ、 <code>TimeProvider</code>

7. セッションの延長のルール（OnCheckSlidingExpiration）

項目	値
説明	/api/system/health へのリクエストは認証の Cookie を延長しない（DEC-019）
戻り値の型	—（Cookie のオプションのイベント）
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

処理概要：Cookie は 2 か所から延長されうるので、両方を扱う（WI-004 DEC-025）。ConfigureApplicationCookie の中で、既存の OnRedirectToLogin / OnRedirectToAccessDenied のハンドラーと並べて：(1) Events.OnCheckSlidingExpiration がヘルスのパスについて ShouldRenew = false にする。(2) Identity の Events.OnValidatePrincipal（30 分の間隔の後に延長するセキュリティスタンプのバリデーター）を包む — それは変更なしで実行され、その後ヘルスのパスについて ShouldRenew = false にするので、取り消されたセッションは引き続き拒否される。Cookie の構成の他の部分に変更しない：有効期限、他のすべてのパスのスライディング、HttpOnly、SameSite=Lax。パスはコントローラーのルートと共有する定数なので、2 つがずれることはない。Cookie のハンドラーの TimeProvider を有効期間の半分を超えて進めた TC-225 で確認する。

可観測性

ai/rules/backend.md に従い、既存の ProductionManagementAI.ProductionOrders の源とメーターを拡張する。

シグナル	名前	種類	属性／バケット
スパン	ProductionOrder.Dashboard	Activity	dashboard.today（工場の日付）、dashboard.active_orders、dashboard.window_completed。失敗時はステータス Error。Program.cs にすでにある Npgsql の計装が文ごとに子のスパンを加え、7 つのクエリそれぞれのコストを示す
メトリクス	pmai.production_orders.dashboard_loaded	カウンター	outcome = success error
スパン	System.Health	Activity	ping が失敗したときステータス Error
メトリクス	pmai.system.health_checks	カウンター	database = ok unavailable — 障害は unavailable の割合の上昇として現れ、件数は開いているダッシュボードの数 ÷ 30 秒で上限がある

ログ：失敗時は例外付きの Error（DashboardSnapshotFailed）。例外の型だけを含む Warning DatabasePingFailed（メッセージは記録しない。接続エラーのメッセージにはホストや資格情報の断片が含まれうる）。成功時は何も記録しない — カウンターとスパンで十分で、ダッシュボードの読み込みは監査対象のイベントではない。数値や製造指示のデータは記録しない。

未解決の決定

なし。動作は 003_BD D-01～D-09、003_DB Q1～Q6、DEC-008、DEC-009、DEC-011、DEC-015 に従う。実装上の注記を 1 つ、決定ではなく記録する：EF Core の

`SqlQuery<T>` が複合の行（例： `DashboardOrderRow` の入れ子の製品）をきれいに対応付けられない場合、行の型は平ら（ `ProductId`、`Sku`、`Name` ）のままにし、マッパーが入れ子にする。仕様は変わらない。